Table des matières

Labo 02 — Développer et faire de l'IA dans Snowflake 1
Prérequis 1
Étape 0 — Poser le contexte 1
Étape 1 — Une fonction Python qui tourne *dans* Snowflake (UDF) 2
Étape 2 — Appeler l'IA de Cortex en SQL 2
Étape 3 (optionnelle) — Une prévision ML sans code de data science 3
Étape 4 (optionnelle) — Un aperçu des Notebooks et de Streamlit 4
Nettoyage 4
Récapitulatif 4

Labo 02 — Développer et faire de l'IA dans Snowflake

Module 2/8 — Formation Snowflake (préparation SnowPro Core, COF-C03)

But du labo — *Voir de vos yeux* le « construire dans Snowflake » du cours : écrire une fonction Python qui s'exécute sur le moteur Snowflake, puis appeler l'IA de Cortex en une simple requête SQL. Tout se fait dans Snowsight (navigateur), sur votre compte d'essai. Comptez 20 à 30 minutes. Objectif = *reconnaître* ces outils, pas devenir développeur Snowpark.

Prérequis

- Le compte d'essai et les objets de base du Cours 00 : entrepôt `SNOWPRO_WH` (XSMALL, `AUTO_SUSPEND`) + base `SNOWPRO_DB`, schéma `SANDBOX`.
- Un worksheet SQL ouvert dans Snowsight (*Projects* → *Worksheets* → *+*).
- Rappel de coût : on reste en `XSMALL`, l'entrepôt s'endort tout seul. Les appels Cortex sont facturés à de minuscules quantités de jetons, quelques cents au grand maximum pour ce labo.

Étape 0 — Poser le contexte

Comme au Cours 01 (section 8), on répond d'abord aux 3 questions. Tout ce qu'on crée ensuite atterrira dans `SNOWPRO_DB.SANDBOX`.

```
USE ROLE SYSADMIN;
USE WAREHOUSE SNOWPRO_WH;
```


© 2026 iAlexMG. Tous droits réservés.

```
USE DATABASE SNOWPRO_DB;
USE SCHEMA SANDBOX;
```

Étape 1 — Une fonction Python qui tourne *dans* Snowflake (UDF)

Théorie : `Theorie_02_SNOW.md`, sections 2 et 3.

On crée une UDF en Python. Le code ne tourne pas sur votre machine : il s'exécute sur l'entrepôt Snowflake.

```
CREATE OR REPLACE FUNCTION etiquette_prix(prix FLOAT)
RETURNS STRING
LANGUAGE PYTHON
RUNTIME_VERSION = '3.11'
HANDLER = 'classer'
AS $$
def classer(prix):
    if prix is None:
        return 'inconnu'
    if prix < 10:
        return 'abordable'
    if prix < 50:
        return 'moyen'
    return 'premium'
$$;
```

Puis on l'utilise dans une requête, exactement comme une fonction SQL native :

```
SELECT valeur, etiquette_prix(valeur) AS categorie
FROM (VALUES (5.0), (25.0), (80.0), (NULL)) AS t(valeur);
```

Ce que vous observez. Trois lignes étiquetées `abordable / moyen / premium`, et `inconnu` pour le `NULL`. Vous venez d'exécuter du Python à l'intérieur de Snowflake : c'est le principe de Snowpark, ici sous sa forme la plus simple (une UDF). *Remarque : le nom `etiquette_prix` n'est pas qualifié parce que le contexte pointe déjà sur `SNOWPRO_DB.SANDBOX` (Cours 01, section 8).*

Étape 2 — Appeler l'IA de Cortex en SQL

Théorie : `Theorie_02_SNOW.md`, section 6.

Aucune infrastructure à monter : les fonctions LLM de Cortex s'appellent comme n'importe quelle fonction SQL.

```
SELECT
  SNOWFLAKE.CORTEX.SENTIMENT('Le service était lent, mais le personnel adorable.')      AS tonalite,
```


© 2026 iAlexMG. Tous droits réservés.

```
  SNOWFLAKE.CORTEX.TRANSLATE('Bonjour, comment allez-vous aujourd''hui ?', 'fr', 'en')  AS en,
  SNOWFLAKE.CORTEX.SUMMARIZE('Snowflake sépare le stockage et le calcul, facture les '
    || 'entrepôts à la seconde, et intègre l''IA via Cortex, appelable directement en SQL.') AS resume;
```

Ce que vous observez. Un score de sentiment (entre −1 et +1), une traduction anglaise, et un résumé, le tout produit par des LLM, sans quitter votre worksheet.

Compte d'essai : cette étape est « à lire », pas à exécuter — Snowflake bloque les fonctions LLM de Cortex (`SENTIMENT`, `TRANSLATE`, `SUMMARIZE`, `COMPLETE`…) sur les comptes *trial*, d'où l'erreur *« AI function … is not available for trial accounts »*. C'est normal, et sans conséquence pour l'examen : le SnowPro Core vérifie que vous reconnaissez qu'un LLM s'invoque en une ligne SQL, pas que vous l'exécutez. Retenez la forme `SNOWFLAKE.CORTEX.<FONCTION>(…)` et passez à la suite. Ce que vous auriez obtenu : un score de sentiment proche de 0 (phrase mitigée), la traduction `Hello, how are you today?` et un résumé d'une phrase.

Ce n'est pas un problème de région — Inutile de lancer `ALTER ACCOUNT SET CORTEX_ENABLED_CROSS_REGION = 'ANY_REGION';` : ce réglage n'agit que sur la disponibilité *régionale* des modèles (utile seulement sur un compte payant dont l'erreur parle de *région* ou de *unknown function*), jamais sur la restriction *compte d'essai*. Vous y perdriez votre temps.

Étape 3 (optionnelle) — Une prévision ML sans code de data science

Théorie : `Theorie_02_SNOW.md`, section 6.

Pour *voir* une fonction ML de Cortex, on fabrique une mini série temporelle puis on demande une prévision. *(Optionnel : la syntaxe des objets ML évolue, alors en cas de doute, gardez le réflexe « ça se fait en SQL, sans coder de modèle » et vérifiez la doc officielle.)*

```
-- une petite série : 14 jours de « ventes »
CREATE OR REPLACE TABLE ventes_jour AS
SELECT DATEADD('day', SEQ4(), '2026-01-01')::DATE AS jour,
       100 + 5 * SEQ4() + UNIFORM(-8, 8, RANDOM())  AS ventes
FROM   TABLE(GENERATOR(ROWCOUNT => 14));

-- entraîner un modèle de prévision sur cette série
CREATE OR REPLACE SNOWFLAKE.ML.FORECAST modele_ventes(
  INPUT_DATA        => SYSTEM$REFERENCE('TABLE', 'ventes_jour'),
  TIMESTAMP_COLNAME => 'jour',
  TARGET_COLNAME    => 'ventes'
);

-- prévoir les 7 prochains jours
CALL modele_ventes!FORECAST(FORECASTING_PERIODS => 7);
```

Ce que vous observez. Sept lignes de prévision (avec un intervalle de confiance), produites sans écrire une ligne de code ML. C'est exactement le rôle des fonctions ML de Cortex (`FORECAST`).

© 2026 iAlexMG. Tous droits réservés.

Étape 4 (optionnelle) — Un aperçu des Notebooks et de Streamlit

Théorie : `Theorie_02_SNOW.md`, sections 4 et 5.

- Notebooks : dans Snowsight, *Projects* → *Notebooks* → *+*. Créez une cellule SQL (`SELECT etiquette_prix(42.0);`) et une cellule Python (`session.sql("SELECT 1").show()`). Vous retrouvez le même moteur, dans un carnet mêlant SQL, Python et texte.
- Streamlit in Snowflake : *Projects* → *Streamlit* → *+* crée une petite appli Python de démonstration, hébergée dans Snowflake. Survolez-la pour *reconnaître* à quoi ça ressemble, pas besoin d'aller plus loin.

Nettoyage

```sql
DROP FUNCTION IF EXISTS etiquette_prix(FLOAT);
DROP TABLE    IF EXISTS ventes_jour;             -- si vous avez fait l'étape 3
-- DROP SNOWFLAKE.ML.FORECAST modele_ventes;     -- idem, pour retirer le modèle
```

(Votre entrepôt `SNOWPRO_WH` s'endort seul via `AUTO_SUSPEND`, rien d'autre à couper.)

Récapitulatif

- ☐ J'ai créé et appelé une UDF Python → du code exécuté dans Snowflake (principe Snowpark).
- ☐ J'ai vu comment appeler un LLM en SQL avec Cortex (sentiment / traduction / résumé), exécuté ou lu si compte d'essai.
- ☐ *(Optionnel)* J'ai obtenu une prévision via une fonction ML de Cortex (`FORECAST`).
- ☐ Je situe Notebooks (carnet SQL/Python) et Streamlit in Snowflake (appli de données).

À retenir : tout ça tourne sur le moteur Snowflake, près des données, avec la gouvernance héritée, l'idée-force du domaine 1, volet développement/IA.

© 2026 iAlexMG. Tous droits réservés.